

Язык Rust (Базовые знания) (Часть 3)

Занятие 5



Кафедра блокчейна

План занятия

Теория + практика (CLI проект)

Лекция (теория)

Разделение на файлы: `crate` / `package` / `mod` / `use`

Обработка ошибок: `Result`, `Option`, оператор `?`

`unwrap` и `expect`: когда можно/нельзя

Операторы: базовые + перегрузка через `traits`

Макросы: `println!`, `vec!`, `format!`, `#[derive]`

Семинар (практика)

Создаём CLI приложение локально (`cargo`)

Разносим код по модулям/файлам

Парсим аргументы

Возвращаем `Result` из функций и используем `?`

Печать ошибок и удобные сообщения

Структурирование кода: `crate`, `package`, `module`

Термины

`Crate` – единица компиляции
(`binary` или `library`)

`Package` – папка с `Cargo.toml`
(может содержать несколько `crates`)

`Module` – пространство имён
внутри `crate` (`mod ...`)

Типичная структура

```
my_pkg/  
Cargo.toml  
src/  
  main.rs    # binary crate entry  
  lib.rs     # library crate entry  
  bar.rs     # just a file  
  
  utils.rs   # module file  
  utils/  
    mod.rs   # module file entry  
    math.rs  
  bar/  
    helper.rs # additional files for bar.rs
```

mod и use: подключаем модули

Как работает

mod объявляет модуль и говорит компилятору, где искать файл
use импортирует путь в текущую область видимости

Пути: crate:: (корень), super:: (вверх), self:: (внутри)

Пример: модуль файлом

```
// src/main.rs
mod utils;
use crate::utils::math::add;

fn main() {
    println!("{}", add(2, 3));
}

// src/utils.rs
pub mod math;

pub fn add(a:i32,b:i32)->i32{a+b}
```

mod и use: подключаем модули

Как работает

`mod` объявляет модуль и говорит компилятору, где искать файл
`use` импортирует путь в текущую область видимости

Пути: `crate::` (корень), `super::` (вверх), `self::` (внутри)

Пример: модуль папкой

```
// src/main.rs
mod utils;
use crate::utils::math::add;

fn main() {
    println!("{}", add(2, 3));
}

// src/utils/mod.rs
pub mod math;

// src/utils/math.rs
pub fn add(a:i32,b:i32)->i32{a+b}
```

pub: видимость и API

Правило по умолчанию

Все элементы `private` по умолчанию `pub` открывает доступ снаружи модуля

`pub(crate)` — видно только внутри `crate`

`pub(super)` — видно родителю

Видимость на практике

```
mod inner {  
    pub(crate) fn internal() {}  
    pub fn public_api() {}  
    fn private_fn() {}  
}  
  
// видно: inner::public_api  
// видно в crate: inner::internal  
// не видно: inner::private_fn
```

super и пути: crate:: / super:: / self::

Когда использовать

`crate::` — абсолютный путь от корня `crate`

`super::` — обратиться к родительскому модулю

`self::` — явность внутри модуля
`use` удобно делать на уровне модуля

super:: пример

```
// src/utils/math.rs
use super::format::fmt;

pub fn
add_and_fmt(a:i32,b:i32)->String{
    fmt(a+b)
}

// src/utils/format.rs
pub fn fmt(x:i32)->String{
    format!("{}", x) }
```

cargo test

Как запускать и писать тесты в Rust

Что делает cargo test

Собирает проект в режиме тестов и запускает все `#[test]`

Юнит-тесты обычно рядом с кодом

Интеграционные тесты — в папке `tests/` (как отдельный crate)

Тест падает через `assert!/assert_eq!/panic!` — это ожидаемо

Полезные команды и атрибуты

`cargo test name_substring` — запустить часть тестов

`cargo test -- --nocapture` — показать `println!` в выводе

`#[should_panic]` — тест ожидает панику (проверяем ошибки)

Практика 1: получаем бинарник (.exe)

Шаги

```
cargo new hello_bin  
cd hello_bin  
cargo run  
Изменяем src/main.rs  
Собираем релиз: cargo build  
--release
```

Команды

```
cargo new hello_bin  
cd hello_bin  
cargo run  
  
# release build  
cargo build --release  
  
# запуск бинарника (путь зависит от ОС)  
./target/release/hello_bin
```

Практика 2: библиотечный crate + модули

Шаги

```
cargo new my_lib --lib
```

```
В src/lib.rs: pub mod utils;
```

```
Создаём src/utils/mod.rs и  
src/utils/math.rs
```

```
Пишем pub fn и подключаем  
через use
```

```
Добавляем тесты: cargo test
```

Мини-пакет

```
# src/lib.rs
pub mod utils;

# src/utils/mod.rs
pub mod math;

# src/utils/math.rs
pub fn add(a:i32,b:i32)->i32{a+b}

# тест (в lib.rs)
#[test]
fn t(){
    assert_eq!(utils::math::add(2,3), 5); }
```

Создаём проекты и модули локально

Что сделаем руками

- 1) `cargo new hello_bin` → бинарный crate (exe)
- 2) `cargo new my_lib --lib` → библиотечный crate
- 3) `cargo add my_lib --path ../my_lib`
- 4) `cargo.toml`: `"my_lib" = { path = "../my_lib" }`
- 5) В корне либы: `pub mod utils; pub fn add()...`
- 6) В приложении: `use my_lib::utils::add;`
- 7) Используем просто `add()`
- 6) Протестируем: `cargo run`

Самостоятельная работа

Библиотека

Реализует умножение, деление,
вычитание

Тестирует

Бинарный файл

Использует библиотеку для
вывода по одному примеру
на экран

Result и Option

Два базовых типа для безопасной логики

Интуиция

Option<T> — значение есть (Some) или нет (None)

Result<T, E> — успех (Ok) или ошибка (Err)

match / if let — безопасное ветвление

Часто: парсинг, поиск, ввод/вывод

Сигнатуры

```
enum Option<T> { Some(T), None }
```

```
enum Result<T, E> {  
    Ok(T),  
    Err(E),  
}
```

Result: паттерны

Возвращаем ошибки наверх, а не прячем

Хорошие практики

Функции возвращают `Result<T, E>`
`main` может вернуть `Result<(), E>`
Ошибки обогащаем контекстом
(exрест/контекст)
Не делаем `silent-failure`, всегда
стараяемся вернуть ошибку

Пример I/O

```
use std::fs;

fn read_cfg(path: &str) ->
std::io::Result<String> {
    fs::read_to_string(path)
}

fn main() -> std::io::Result<()> {
    let cfg = read_cfg("config.txt");
    println!("{}", cfg);
    Ok(())
}
```

Option: паттерны

Нет значения — это не ошибка, а состояние

Частые приёмы

match / if let / while let

unwrap_or / unwrap_or_else / map /
and_then

Option → Result через ok_or / ok_or_else

Полезно для поиска и конфигов

Option → Result

```
fn get_user(id: u64) -> Option<String> {  
    None  
}  
  
fn must_get(id: u64) -> Result<String,  
&'static str> {  
    get_user(id).ok_or("user not found")  
}
```

Оператор ?

Ранний выход при ошибке

Что делает ?

Для Result: если Err → возвращаем Err наружу

Для Option: если None → возвращаем None наружу

Сокращает код и повышает читаемость
Требует совместимый возвращаемый тип

До / После

```
// без ?  
let f = match std::fs::File::open(p) {  
    Ok(v) => v,  
    Err(e) => return Err(e),  
};  
  
// с ?  
let f = std::fs::File::open(p)?;
```

Разберем unwrap

Быстро, но опасно: когда можно и чем заменить

Что делает unwrap()

Option: Some(x) → x, None → panic!

Result: Ok(x) → x, Err(e) → panic! (печатает e)

Удобно в примерах, прототипах и тестах

В CLI/проде лучше: обработать ошибку и дать понятное сообщение

Когда допустимо

В тестах и учебных примерах
(показать идею быстро)

Когда есть инвариант: «сюда Err/None
попасть не может»

Если хотите упасть сразу при плохом
конфиге (осознанно)

Но в обычном потоке: Result + ? или
match/if let

```
let n: i32 = "42".parse().unwrap();  
let n: i32 = "42".parse()?; // когда лучше прокинуть ошибку вверх, а не умереть здесь и сейчас
```

Разберем expect

expect = unwrap с текстом ошибки

Когда допустимо

В прототипе, тестах, примерах

В случаях «это невозможно» (инвариант)

expect лучше unwrap: даёт контекст

В CLI лучше печатать ошибку и выходить корректно

Пример

```
use std::env;

let port: u16 = env::var("PORT")
    .expect("PORT is not set")
    .parse()
    .expect("PORT must be a number");
```

Задание

Парсер

Создать функцию, которая парсит число
либо выдает ошибку

```
fn parse_int(s: String) -> Result<i32,  
String>
```

```
let num = s.parse::<i32>()
```

```
expect("message")
```

Замыкания и цепочки

«Лямбда» функции: `map` / `filter` / `and_then`

Что такое замыкание (closure)

Синтаксис: `|x| x + 1` (аналог «лямбды»)

Может «захватывать» переменные снаружи

Часто используется в `map/filter/and_then` и итераторах

Зачем это в Rust

Пишем цепочки без `match`-спагетти

Выражаем «что сделать» коротко и читаемо

Ошибки/`None` проходят по цепочке автоматически

Примеры (`Option`/`Result`/`Iterator`)

```
let factor = 2;
let opt: Option<i32> = Some(7);
let out = opt
    .map(|x| x * factor)           // Some(14)
    .filter(|x| *x > 10)          // Some(14) или None
    .and_then(|x| x.checked_add(1)); // Option<i32>
```

Совет: сначала делайте код понятным, потом «сжимайте» цепочками.

Задание

Простое

Сделать вектор с элементами 0, 3, 8, 4, 9, 1, 4

С помощью `map` умножить каждый элемент на 2

Вывести новый массив на экран

Сложнее

(продолжаем код после простой задачи)

С помощью `filter` оставить те элементы, которые после умножения больше 7

Вывести новый массив на экран

Сцепляем Result и Option

Пишем понятный контроль потока

Полезные методы

ok_or_else — превращаем Option в Result

map_err — преобразуем ошибку в свой тип

Держите функции маленькими и явными

map_err + ?

```
fn parse_num(s: &str) -> Result<i32, String> {
    s.parse::<i32>().map_err(|e|
        e.to_string())
}

fn do_work(s: Option<&str>) ->
    Result<i32, String> {
    let s = s.ok_or_else(|| "missing
input".to_string());
    let n = parse_num(s)?;
    Ok(n * 2)
}
```

ref и ref mut

Заимствование внутри паттернов (match/let)

Зачем нужен ref

В паттернах по умолчанию значения могут «двигаться» (move)
ref говорит: «свяжи переменную как ссылку (&), не забирай владение»
ref mut — то же самое, но как &mut (можно менять через *var)

Практические подсказки

Если матчите по ссылке: match &value { ... } — ref часто не нужен
Для Option/Result есть as_ref()/as_mut() (удобно в цепочках)
ref удобен при распаковке struct/enum, чтобы не перемещать поля

```
struct User { name: String, count: u32 }

let mut u = User { name: "Ann".into(), count: 0 };
let User { ref name, ref mut count } = u; // borrow поля
println!("{}", name);
*count += 1;
println!("{}", u.count);
```

trait (трейты)

Интерфейсы поведения: что объект «умеет делать»

Что такое trait

Trait — набор методов: контракт поведения (как «interface»)

Тип может реализовать много trait'ов (композиция вместо наследования)

В стандартной библиотеке: Debug, Display, Iterator, Clone, Copy...

В Solana/Anchor трейты часто дают удобные абстракции и расширения
derive помогает авто-реализациям (`#[derive(Debug, Clone)]`)

```
trait Area { fn area(&self) -> f64; }

struct Circle { r: f64 }
impl Area for Circle {
    fn area(&self) -> f64 { 3.1415 * self.r * self.r }
}

fn print_area(x: &impl Area) { println!("{}", x.area()); }
```

Операторы в Rust

Арифметика, сравнения, логика, диапазоны

Базовые категории

+ - * / % (арифметика)

== != < <= > >= (сравнение)

&& || ! (логика) • & | ^ << >> (битовые)

.. и ..= (диапазоны: for, match, slices)

as (приведение типов)

Диапазоны

```
for i in 0..3 { /* 0,1,2 */ }  
for i in 0..=3 { /* 0..3 */ }
```

```
enum x {  
    Var1  
}
```

```
match x {  
    0..=9 => println!("digit"),  
    _ => println!("other"),  
}
```

Перегрузка операторов

Через traits из `std::ops`

Идея

Операторы — сахар над trait-методами
`a + b` вызывает `Add::add(a, b)`
Полезно для математических типов
Не злоупотребляйте — читаемость важнее

`std::ops::Add`

```
use std::ops::Add;

#[derive(Clone, Copy)]
struct Vec2 { x: i32, y: i32 }

impl Add for Vec2 {
    type Output = Vec2;
    fn add(self, rhs: Vec2) -> Vec2 {
        Vec2 { x: self.x + rhs.x, y:
self.y + rhs.y }
    }
}

let c = Vec2{x:1,y:2} + Vec2{x:3,y:4};
```

Макросы

Код, который генерирует код

Зачем нужны

Уменьшают boilerplate макросы (println!, vec!, format!)

Позволяют DSL-подобный синтаксис
Работают до компиляции (macro expansion)

Типы проверяются после расширения макроса

Примеры

```
println!("{}", 10);  
let v = vec![1, 2, 3];  
let s = format!("hi {}", 42);
```

```
#[derive(Debug, Clone)]  
struct User { id: u64 }
```

macro_rules!

Декларативные макросы

Интуиция

Паттерны → шаблоны генерации кода
Удобно для повторяющихся конструкций
Отладка сложнее, чем у функций

Мини-макрос

```
macro_rules! say {  
    ($msg:expr) => {  
        println!("{}", $msg);  
    };  
}  
  
say!("hello");
```

Семинар: первый CLI проект

Командная строка + модули + Result/?

Что соберём

`cargo new my_cli`

Команда: `my_cli <path>` (читает файл и выводит статистику)

Модули: `cli.rs`, `io.rs`, `stats.rs`, `error.rs`

Result + ? в каждом слое

Печать ошибок: `eprintln!` + exit code

Команды

```
cargo new my_cli
cd my_cli
cargo run -- data.txt
```

```
# структура
src/main.rs
src/cli.rs
src/io.rs
src/stats.rs
src/error.rs
```

Парсер CLI: пример кода

Напишите недостающие модули

```
mod cli;
mod error;
mod io;
mod stats;

use error::AppError;

fn main() -> Result<(), AppError> {
    let args = cli::parse_args()?;
    let text = io::read_to_string(&args.path)?;
    let out = stats::summarize(&text);
    println!("{out}");
    Ok(())
}
```